

Notas de ICC 2



Ariel Alves da Silva



Academic year 2026

Índice

Índice	2
1. Análise de Algoritmos I	4
1.1. Notações	4
1.2. Exercício	5
1.3. Escolhendo $T(n)$: Melhor, pior, média	5
1.4. Exercício: maxSubArray	5
1.5. Cálculo de complexidade para sub-rotinas recursivas	5
Método da substituição	6
Método da Árvore de recursão	7
Método Mestre	7
Exercícios	8
2. Métodos de Ordenação	10
2.1. Bubble Sort	10
Lógica	10
Complexidade	10
2.2. Quick Sort	10
Lógica	10
Complexidade	10
2.3. Insertion Sort	11
Lógica	11
Complexidade	11
2.4. Shell-Sort	11
Lógica	11
Complexidade	11
2.5. Seleção Direta	11
Complexidade	12
2.6. Heap Sort	12
Lógica	12
Complexidade	12
2.7. Radix Sort	13
Lógica	13

2.8. Comparação entre os métodos mais conhecidos	13
Ordem aleatória dos elementos	13
Array já ordenado	13
Array ordenado inversamente	14
2.9. Conclusão	14

1. Análise de Algoritmos I

Buscamos formas de comparar algoritmos de forma independente de Hardware, Linguagem de Programação e Habilidade do Programador. Comparamos algoritmos, e não programas.

1.1. Notações

A função $T(n)$ é o custo real.

- $T(n) = \mathcal{O}(f(n)) \Rightarrow T(n) \leq c \cdot f(n)$, p/ $n > n_0$ - Upper Bound. Maior ou igual ao custo real da função.
($f(n) = n^2$, por ex)
- $T(n) = \Omega(f(n)) \Rightarrow T(n) \geq c \cdot f(n)$, p/ $n > n_0$ - Lower Bound. Menor ou igual ao custo real da função.
- $T(n) = \Theta(f(n)) \Rightarrow \geq c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$, p/ $n > n_0$ - Intermediário.

Na função $f(n)$ não se costuma incluir constantes ou termos de menor ordem.

Ex: $T(n) = 3n + 3$ (nossa função)

$$T(n) = \mathcal{O}(n)?$$

$$3n + 3 \leq c \cdot n$$

$$c \in \mathbb{N}, c \geq 4$$

Provado que $T(n)$ é $\mathcal{O}(n)$.

$$T(n) = \mathcal{O}(\log(n))?$$

$$3n + 3 \leq c \cdot \log(n)$$

Não existe c tal que satisfaça a condição acima. Provado que $T(n) \neq \mathcal{O}(\log(n))$.

$$T(n) = \Omega(n)?$$

$$3n + 3 \geq c \cdot n$$

$$c \in \mathbb{N}^*, c \leq 3$$

Provado que $T(n)$ é $\Omega(n)$.

$$T(n) = \mathcal{O}(\log(n))?$$

$$3n + 3 \geq c \cdot \log(n)$$

$$c \in \mathbb{N}^*$$

Provado que $T(n)$ é $\Omega(\log(n))$.

1.2. Exercício

$$f(n) = n^{1.5} \stackrel{/n}{\Rightarrow} n^{0.5} \stackrel{\wedge_2}{\Rightarrow} n$$

$$g(n) = n \log(n) \stackrel{/n}{\Rightarrow} \log(n) \stackrel{\wedge_2}{\Rightarrow} \log^2(n)$$

Para comparar, manipulamos de acordo a chegar nos algoritmos da tabela.

$$c \leq \log \leq \log^2 \leq n \leq n \log(n) \leq n^2 \leq n^3 \leq 2^n$$

1.3. Escolhendo $T(n)$: Melhor, pior, média

A função $T(n)$ pode ser o melhor, pior ou melhor caso.

$$T_{\text{melhor}(n)} \leq T_{\text{média}(n)} \leq T_{\text{pior}(n)}$$

1.4. Exercício: maxSubArray

Divisão e conquista:

-2 11 -4 13 -5 -2 -2 11 -4 13 | -5 -2 -2 11 | -4 13 | -5 -2 -> Retorna 11 | 13 | 0

1.5. Cálculo de complexidade para sub-rotinas recursivas

É necessário recorrer à análise de recorrência: Equação ou desigualdade que descreve uma função em termos de seu valor em entradas menores.

Ex: Fibonacci

- $f(0)=0, f(1)=1, f(i)=f(i-1)+f(i-2)$

```
sub-rotina fib(n: numérico)
início
declare aux numérico;
se n≤1
    então aux←1
    senão aux←fib(n-1)+fib(n-2);
retorne aux;
fim
```

Caso 1: $n = 0, n = 1 \rightarrow T(0) = T(1) = 3$

Caso 2: $n \geq 2 \rightarrow T(n) = T(n-1) + T(n-2) + 6$

$\{T(0)=T(1)=3, T(n)=T(n-1)+T(n-2)+6 \mid n \geq 2\}$

Método da substituição

Supõe-se (aleatoriamente ou com base em experiência) um limite superior para a função e verifica-se se ela não extrapola esse limite (uso de indução matemática)

Hipótese: $O(2^n)$

$$T(n) = O(2^n) \rightarrow T(n) \leq c \cdot 2^n, c > 0$$

Base da indução:

$$T(0) = 3 \leq c \cdot 2^0$$

$$T(1) = 3 \leq c \cdot 2^1$$

$$T(k-1) \leq c \cdot 2^{k-1}$$

$$T(k-2) \leq c \cdot 2^{k-2}$$

Mostrar que $T(k) \leq c \cdot 2^k$ substituindo:

$$T(k) = T(k-1) + T(k-2) + 6$$

$$T(k) \leq c \cdot 2^{k-1} + c \cdot 2^{k-2} + 6$$

$$T(k) \leq c \cdot 2 \cdot 2^{k-2} + c \cdot 2^{k-2} + 6$$

$$T(k) \leq c \cdot 3 \cdot 2^{k-2} + 6$$

$$T(k) \leq c \cdot \left(\frac{3}{2^2}\right) 2^k + 6$$

Necessário encontrar uma constante que satisfaça a desigualdade:

$$c \cdot \left(\frac{3}{2^2}\right) 2^k + 6 \leq c \cdot 2^k$$

$$c \cdot 2^k \cdot \left(\frac{3}{4} - 1\right) \leq -6$$

$$c \leq -\frac{6}{2^k \cdot \left(\frac{3}{4} - 1\right)}$$

$$c \geq \frac{24}{2^k}$$

Menor caso: $k = 0 \rightarrow \frac{24}{2^k} = 24$. Portanto, $\exists c \in \mathbb{R}$ t.q. $c \geq \frac{24}{2^k}$

Hipótese: $O(n^2)$

Base da indução:

$$T(0) = 3 \leq c \cdot 0^2$$

$$T(1) = 3 \leq c \cdot 1^2$$

$$T(k-1) \leq c \cdot (k-1)^2$$

$$T(k-2) \leq c \cdot (k-2)^2$$

Mostrar que $T(k) \leq c \cdot n^2$ substituindo:

$$T(k) = T(k-1) + T(k-2) + 6$$

$$T(k) \leq c \cdot (k-1)^2 + c \cdot (k-2)^2 + 6$$

$$T(k) \leq c \cdot [k^2 - 2k + 1 + k^2 - 4k + 4] + 6$$

$$T(k) \leq c \cdot [2k^2 - 6k + 5] + 6$$

Logo:

$$c \cdot [2k^2 - 6k + 5] + 6 \leq c \cdot k^2$$

$$c \cdot [-k^2 + 6k - 5] \geq 6$$

Não existe c fixo que satisfaça a desigualdade para qualquer k .

$$c \geq \frac{6}{-k^2 + 6k - 5}$$

Obs: ao dividir por um número negativo, o sentido da desigualdade inverte (continua não sendo válido)

Método da Árvore de recursão

$T(n) \rightarrow c, T(n/2) \rightarrow c, c, T(n/4) \rightarrow c, \dots, c$

Altura = $\log n + 1$

Método Mestre

Teorema: Sejam $a \geq 1$ e $b > 1$ constantes, seja $f(n)$ uma função e seja $T(n)$ definida como:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Então $T(n)$ pode ser limitado assintoticamente como:

- Se $f(n) = O(n^{\log_b(a)-x})$ para algum $x > 0$, então $T(n) = \Theta(n^{\log_b a})$
- Se $f(n) = \Theta(n^{\log_b a})$, então $T(n) = \Theta(n^{\log_b a} \log(n))$
- Se $f(n) = \Omega(n^{\log_b(a)+x})$ para algum $x > 0$, e se $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ para algum $c < 1$ e para todo n suficientemente grande, então $T(n) = \Theta(f(n))$

Ex: $T(n) = 9T\left(\frac{n}{3}\right) + n$

- $a = 9$
- $b = 3$
- $f(n) = n$
- $n^{\log_b a} = n^{\log_3 9} = n^2$

Tentamos o caso 1:

$$f(n) = O(n^{2-x})$$

$$n = O(n^{2-x})$$

$$n = \mathcal{O}(n)$$

$$n \leq c \cdot n$$

Vemos que o primeiro caso é satisfeito. Logo:

$$T(n) = \Theta(n^2)$$

Ex:

$$T(n) = T\left(\frac{2n}{3}\right) + 1$$

- $a = 1$
- $b = \frac{3}{2}$
- $f(n) = 1$
- $n^{\log_b a} = n^0 = 1$

Tentamos o caso 1:

$$f(n) = \mathcal{O}(n^{0-x})$$

$$n^0 \leq c \cdot n^{0-x}$$

O caso 1 não é válido.

Tentamos o caso 2:

$$f(n) = \Theta(n^0)$$

$$1 = \Theta(\dots)$$

???

Exercícios

1) $T(n) = 4T\left(\frac{n}{2}\right) + n$

$$a = 4, b = 2, f(n) = n, n^{\log_b a} = n^{\log_2 4} = 2$$

Caso 1:

$$n = \mathcal{O}(n^{\log_2(4)-x})$$

$$n = \mathcal{O}(n^{2-x})$$

$$n = \mathcal{O}(n) \rightarrow n \leq c \cdot n$$

Logo:

$$T(n) = \Theta(n^2)$$

2) $T(n) = 4T\left(\frac{n}{2}\right) + n^2$

Caso 1:

$$n^2 = \mathcal{O}(n^{\log_2(4)-x})$$

$$n^2 = \mathcal{O}(n^{2-x})$$

Não existe $x > 0$ que satisfaça isso.

Caso 2:

$$n^2 = \Theta(n^{\log_2(4)})$$

$$n^2 = \Theta(n^2)$$

$$c_1 \cdot n^2 \leq n^2 \leq c_2 \cdot n^2$$

Isso é satisfeito. Logo:

$$T(n) = \Theta(n^2 \log(n))$$

$$3) T(n) = 4T\left(\frac{n}{2}\right) + n^3$$

Caso 3:

$$n^3 = \Omega(n^{\log_2 4+x})$$

$$n^3 = \Omega(n^{2+x})$$

$$n^3 = \Omega(n^3) \Rightarrow c \cdot n^3 \leq n^3$$

Verificando segunda condição:

$$a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$$

$$4\left(\frac{n}{2}\right)^3 \leq c \cdot n^3$$

$$\frac{1}{2} \leq c \leq 1$$

Logo, $T(n) = \Theta(n^3)$

$$4) T(n) = 2T\left(\frac{n}{2}\right) + n^3$$

$$5) T(n) = T\left(\frac{9n}{10}\right) + n$$

2. Métodos de Ordenação

2.1. Bubble Sort

Lógica

Percorrer o array várias vezes, trocando elementos consecutivos não ordenados, até que não haja mais trocas.

Complexidade

2 for aninhados. Complexidade: $O(n^2)$

2.2. Quick Sort

Lógica

Escolhe um pivô, coloca menores que ele à esquerda, maiores que ele à direita, divide e conquista.

1. Percorre esquerda até `v[i] >= x`
2. Percorre direita até `v[i] <= x`
3. Troca `v[i]` , `v[j]`
4. Continua incrementando até `i==j`
5. Divide e conquista

Complexidade

Se o pivô é o maior ou menor elemento:

$$T(n) = T(n-1) + O(n) \Rightarrow O(n^2)$$

Se a escolha do pivô divide o vetor em partes iguais:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \Rightarrow O(n \log_2(n))$$

Caso médio (Sedgewick e Flajelot, 1996)

$$T(n) \approx 1.386n \log_2(n) - 0.846n = O(n \log_2 n)$$

2.3. Insertion Sort

Lógica

Criar um subarray à esquerda. Vamos inserir, um a um, os elementos à direita nesse subarray à esquerda.

```
[10] 2 7 5  
[2 10] 7 5  
[2 7 10] 5  
[2 5 7 10]
```

Complexidade

$\mathcal{O}(n^2)$ no pior caso. $\mathcal{O}(n)$ no melhor caso.

2.4. Shell-Sort

Lógica

Dividir o vetor em subarrays de elementos de distância h . Aplica-se insertion sort a cada um, sendo que h é reduzido sucessivamente.

```
h = 2  
[10] 2 [7] 5  
[7] 2 [10] 5  
7 [2] 10 [5]  
7 2 10 5  
  
h = 1  
[7] 2 10 5  
[2 7] 10 5  
[2 5 7] 10
```

Complexidade

Com uma sequência adequada de incrementos de h , shell-sort é aproximadamente $\mathcal{O}(n(\log n)^2)$

Knuth (1973) sugere: define uma função recursiva h tal que $h(1) = 1$ e $h(i + 1) = 3 * h(i) + 1$

Complexidade: $\mathcal{O}(n^{\frac{3}{2}})$

2.5. Seleção Direta

Busca o menor elemento e o coloca na posição correta, da esquerda pra direita.

Complexidade

$\mathcal{O}(n^2)$

2.6. Heap Sort

Uma heap é uma estrutura de dados em que há uma ordenação entre elementos. A representação é via árvore binária.

Heap max: pai maior que filhos Heap min: pai menor que filhos

- Nó esquerdo: menor entre 2 nós
- Nó direita: maior entre 2 nós

Um heap pode ser representado por um vetor.

- Pai do nó k : $\frac{k-1}{2}$ (inteiro)
- Filho esquerdo: $2k + 1$
- Filho direito: $2k + 2$
- Folhas de $\frac{n}{2}$ em diante (nós sem filhos).

Assume-se que:

- A raiz está sempre na posição 0 do vetor.
- comprimento(vetor) indica o número de elementos do vetor.
- tamanho_heap(vetor) indica o número de elementos no heap armazenado no vetor

```
[Pai, LFilho1, RFilho1, LFilho2, RFilho2, ...]
```

Lógica

1. Construir um max heap com base no vetor desordenado
2. Troca-se a raiz (maior elemento) com o último elemento e rearranja-se o heap, utilizando somente o que sobrou (último elemento ordenado)
3. Diminuir tamanho do heap em 1
4. Rearranjar max heap, se necessário
5. Repetir processo $n-1$ vezes

Complexidade

- O nível mais baixo tem aprox. $n/2$ nós e não precisam ser analisados.
- O segundo nível mais baixo tem aprox. $n/4$ nós e precisam de no máximo 1 atualização.
- O terceiro nível mais baixo tem aprox $n/8$ nós e precisam de no máximo 2 atualizações.

$$\text{custo_total} = \frac{n}{2} \cdot 0 + \frac{n}{4} \cdot 1 + \frac{n}{8} \cdot 2 + \dots + 1 \cdot \log_2 n$$

$$\text{custo_total} = n \cdot \sum_{i=0}^{\log_2(n-1)} \frac{i}{2^{i+1}} = \frac{n}{2} \sum_{i=0}^{\log_2(n-1)} \frac{i}{2^i}$$

2.7. Radix Sort

Lógica

Os números são ordenados por seus dígitos, dos menos significativos para os mais significativos. Ex: ordenar 236 e 235 por 6 e 5. Se não bastar, comparam-se os do meio, etc.

Utilizam-se listas

- Uma fila para cada dígito
- Os números vão sendo inseridos na fila de acordo com o dígito sendo avaliado
- A cada iteração, os números estão mais próximos da ordenação final

2.8. Comparação entre os métodos mais conhecidos

Ordem aleatória dos elementos

	500	5.000	10.000	30.000
Inserção	11,3	87	161	—
Seleção	16,2	124	228	—
<i>Shellsort</i>	1,2	1,6	1,7	2
<i>Quicksort</i>	1	1	1	1
<i>Heapsort</i>	1,5	1,6	1,6	1,6

Array já ordenado

	500	5.000	10.000	30.000
Inserção	1	1	1	1
Seleção	128	1.524	3.066	—
<i>Shellsort</i>	3,9	6,8	7,3	8,1
<i>Quicksort</i>	4,1	6,3	6,8	7,1
<i>Heapsort</i>	12,2	20,8	22,4	24,6

Array ordenado inversamente

	500	5.000	10.000	30.000
Inserção	40,3	305	575	—
Seleção	29,3	221	417	—
<i>Shellsort</i>	1,5	1,5	1,6	1,6
<i>Quicksort</i>	1	1	1	1
<i>Heapsort</i>	2,5	2,7	2,7	2,9

2.9. Conclusão

- Inserção direta: interessante para arquivos com menos de 20 elementos, estável.
- Seleção: Só é vantajoso quanto ao número de movimentos de registros: $O(n)$. Pode ser usado para arquivos < 1000
- Shell-sort: Eficiente para arquivos de tamanho moderado. Não possui pior caso ruim.
- Heap-sort: Executa sempre em $O(n \log n)$. Não precisa de memória adicional.
- Merge-sort: Executa sempre em $O(n \log n)$. Precisa de memória adicional.
- Quick-sort: Mais eficiente que existe para situações variadas. Recursividade exige memória adicional. Para escolha do pivô, usar mediana de três elementos.